

P1376R0

Summary of freestanding evening session discussions

Published Proposal, 2018-11-24

This version:

<http://wg21.link/P1376R0>

Author:

[Ben Craig](#) (National Instruments)

Audience:

WG21

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/freestanding_proposal/blob/master/san_diego_evening.bs

Abstract

Summary of discussion in San Diego 2018 evening session discussing freestanding direction

Table of Contents

- 1 **Freestanding is...**
- 2 **Assorted statements from the evening**
- 3 **Discussions with regards to modules**
- 4 **Toolchains**
- 5 **Related Polls**

- 5.1 SG1 polls for [P1105R1] (Nov 7, 2018)
- 5.2 EWG polls for [P1212R0] (Nov 6, 2018)
- 5.3 SG14 cppcon meeting polls for D1105R1.3 (Sep 26, 2018)
- 5.4 SG14 telecon polls for [P1105R0] (July 11, 2018)
- 5.5 LEWG poll for [P0829R0] (Nov 8, 2017)
- 5.6 LEWG poll for [P0581R1] (Mar 13, 2018)

References

Informative References

§ 1. Freestanding is...

Throughout the evening session, many potential qualities of freestanding were pitched. Many of these potential qualities are mutually exclusive, and lead to drastically different designs.

- Freestanding C++ is the C++ that allows you to swap the `std::` library.
- Freestanding C++ is the C++ you can have w/o an OS.
- Freestanding C++ is the C++ you can have inside an OS.
- Freestanding C++ is the C++ you get with always-LTO (optimizes out what you don't use).
- Freestanding C++ is the C++ is statically linked (and therefore has no ABI restrictions).
- Freestanding C++ is the C++ used in many (different) real-world applications that C++ ignores.
- Freestanding C++ is the C++ without overheads your platform can't support.
- Freestanding C++ really has you not-pay for what you not-use.
- Freestanding C++ is the C++ I use in my program (different from /other/ program).
- Freestanding C++ is the C++ that runs on accelerators / alternate hardware.
- Freestanding C++ is the C++ root dialect in the family of C++ dialects.
- Freestanding C++ is the minimum bar to reach before you are C++.
- Freestanding C++ is the C++ I can teach my C friends.
- Freestanding is the C++ that I can use in C-files.
- A freestanding implementation is one in which execution may take place without the benefit of the operating system.
- Freestanding C++ is the (nearly) minimal subset of the library needed to support all the language features.

- Freestanding C++ is the C++ that can run on any environment (e.g. any chip, any OS, any browser, any GPU, etc).
- Freestanding C++ is the C++ that can run on any environment from the last X years.
- Freestanding C++ is the C++ that can run on any environment that is "reasonable".
- Freestanding C++ is the signal / interrupt safe subset of C++.
- Freestanding C++ is the C++ for systems with 8 bytes of RAM. 8 KB of RAM. 8 MB of RAM.
- Freestanding C++ is the C++ for systems that support at least operation X.
- The freestanding library should be implementable with the freestanding language + assembly + intrinsics.
- The OS should be implementable with freestanding C++ + assembly + intrinsics.
- Hosted C++ should be implementable with the OS + freestanding C++ + assembly + intrinsics.
- There is no freestanding C++ with exceptions.
- Freestanding is a consensual delusion.
- Freestanding should require no special dispensation from the operating environment.
- Freestanding is the subset of C++ that leaves no room for a lower level language.

§ 2. Assorted statements from the evening

- Current freestanding C++ does not meet anyone's needs
- Design intent behind Freestanding is that your standard library could be swapped in
 - Counterpoint: That is not true
 - Where is this design philosophy written?
- Say "subset" and you'll never get changes. What you want is to write programs that don't have implementation-imposed overhead or things that your platform does not support. What you want is for the implementation to only put things into the program that are needed to run the program. Goal should be for locally supported subsets to work. Compilers are getting smarter and can spot this stuff.
 - Agreement with previous statement, particularly with LTO. LTO can figure out what can be omitted.
- We will get nothing so long as we insist on calling what we want freestanding.
- Unofficial subset that should be brought back into C++.

- We should stop making special rules and bring freestanding into the language.
- Freestanding users disagree about what they want.
- Committee needs to admit that `-fnoexceptions` exists.
- A module attribute to give a compile error when exceptions are used would be helpful.
- We are all using freestanding if `-fnoexceptions` is in place.
- Freestanding does not currently include `memcpy`, `std::move`, or algorithms.
- Initialization of program statics is not defined in terms of `main`.
- Freestanding implementations would need to produce a lot more documentation.
- Users choose not to use conforming toolchains.
- None of the major OS kernels enable conforming C++98 environments for their drivers (freestanding or otherwise)
 - My platform does
 - What about `uncaught_exception()` ?
- I don't know if my environment is freestanding, nor can I find out.
- Do we need a choose-your-own-set menu of features for freestanding C++?
- C++ freestanding needs RTTI, exceptions etc. C has no complex numbers in freestanding.
 - C11 does require `_Thread_local`
- Requiring LTO is not a solution. Non-conforming C++ is a lower level language
- What are the target applications? Merely resource-constrained? Partial OS support?
- Do we aim for the lowest common denominator? Probably not.
- What should we consider to be "reasonable" hardware?
- Do we want a freestanding library to work in a hosted environment?
- Should OS changes with the same ABI and no calls into OS function be portable without rebuilding?
- Should ubiquitous vendor extensions be standardized?
- Someone should be able to write a kernel driver in freestanding C++
- There are Linux kernel files that deliberately define variables named after C++ keywords
- This could be a fork of the language - do not constrain your ambition.

- Splitting C++ from C was a bad idea. Spinning freestanding off as another language would be an even worse idea
- I have no idea who freestanding users are.
 - I hear from my freestanding users.
- Embedded developers are afraid of C++ (education issues)

§ 3. Discussions with regards to modules

- Modules are probably happening for C++20. Proposal for library modules reorganizes standard library. Needs attention.
- Implementers have looked at the paper and had input.
- Modules will be useful even if the standard library is not modularized.
- Implementers will have a chance to interact with users (including freestanding users)
- Don't want a reorganization to be linked to freestanding or to C++20.
- Some implementers may have tried doing the reorganization, but not all implementers have tried it.
- When metaclasses first came to C++, there was concern that it would fracture the language and introduce dialects. People do not like the appearance of variants of C++.
- If you put in `-fnoexceptions`, it's not a dialect as the code can't build if it throws.
- Some classes throw in constructors.
- There are already libraries designed for cases with and without exceptions.
- What would the tolerable cost of exceptions be?
 - Any size or runtime cost is intolerable.
 - Cost proportional to existing program size is no good.
 - Cost proportional to use of exceptions might be tolerable.
- Let's not focus on exceptions. We can continue the lie. `-fnoexceptions` solves this problem already.
- Introducing exceptions removes status code checks. What's the size difference between code with exceptions and code with status code checks?
- My company periodically builds with `-fnoexceptions`. Handling of exception specifications on virtual functions differs between compilers.

- C++ is bad at code size
 - Not true. Compare to C it is good. C++ code vs. C-code has smaller footprint and runs faster.
 - Maybe you are just better programmers?
- C++ committee works with constexpr which has no exception support.
- There are language features in C++ (like templates) that lead to better optimizations than C with macros.
- Reorganizing the standard library will lead to reorganizing freestanding.

§ 4. Toolchains

- Editor debugger, code generator come from compiler vendor.
- Linker map comes from chip vendor. Linker map won't work with C++ out of the box.
- Start code is provided by chip vendor.
- Toolchain as a whole is partially from chip vendor, partially from compiler vendor.
- Chip manufacturers don't want to support C++.
- A better freestanding implementation is not sufficient. We use flags and LTO.
- Tool vendors and chip vendors are openly hostile to C++.
- There is a future version of C++ where users and chip vendors are working together.
- With flags and LTO, is what you have usable?
 - Yes. With C++14 it's awesome. It could be better.
- The linker map startup code cannot be fixed by the committee.
- Is there anything we've said that you think would mean that toolchain vendors would refuse to support C++?
- I don't think it's their problem. We need to deal with people turning off exceptions and ignoring it.
- We'd need to do design work to be able to build without RTTI and exceptions. We'd also want to exclude locale... the list is big.
- Inclusion of headers by other headers can cause builds to fail.
- If we rejuvenate C++ freestanding as important, library implementers can tidy up their libraries.

§ 5. Related Polls

§ 5.1. SG1 polls for [\[P1105R1\]](#) (Nov 7, 2018)

Request the wider WG21 group to form an overarching direction for freestanding

SF/F/N/A/SA

18/3/1/0/0

Conforming freestanding implementations could make `thread_local` ill-formed

SF/F/N/A/SA

3/12/5/0/0

Conforming freestanding implementations could omit lock-free atomics

SF/F/N/A/SA

2/6/4/6/2

Conforming freestanding implementations could omit thread-safe statics

SF/F/N/A/SA

0/6/8/4/2

§ 5.2. EWG polls for [\[P1212R0\]](#) (Nov 6, 2018)

Subset the language support in the standard library modules paper so that the parts requiring an OS are not freestanding

SF/F/N/A/SA

3/2/5/11/3

Expand the library, but maintain all language support facilities in standard library modules

SF/F/N/A/SA

3/10/5/0/1

§ 5.3. SG14 cppcon meeting polls for D1105R1.3 (Sep 26, 2018)

Minutes

Poll: I want to know if we're on board with a way to disable dynamic, type-based exceptions (this proposal is neutral with respect to static exceptions)

(no opposition in this room)

§ 5.4. SG14 telecon polls for [\[P1105R0\]](#) (July 11, 2018)

Minutes

Poll 1: get rid of freestanding

SF/F/N/A/SA

0/1/2/9/11

Poll 2: modify freestanding along the lines of [\[P1105R0\]](#), encouragement for further work, agree with most of it

SF/F/N/A/SA

5/13/4/0/0

§ 5.5. LEWG poll for [\[P0829R0\]](#) (Nov 8, 2017)

Minutes

Change the definition of freestanding along these lines

SF/F/N/A/SA

1/7/7/1/0

Add a classification for embedded-friendly interfaces

SF/F/N/A/SA

1/5/9/1/0

We support proposed removal of typeinfo and exception headers from freestanding

SF/F/N/A/SA

1/6/6/1/2

We are ok with marking freestanding support on the per-API level, not per-header

SF/F/N/A/SA

0/15/0/3/0

We are ok with marking freestanding support on a method level, not per-class.

SF/F/N/A/SA

0/9/3/5/1

§ 5.6. LEWG poll for [\[P0581R1\]](#) (Mar 13, 2018)

Minutes

We want to see more work on defining module(s) specifically targeting basic/freestanding.
UNANIMOUS CONSENT

§ References

§ Informative References

[P0581R1]

Marshall Clow; et al. [Standard Library Modules](http://wg21.link/P0581R1). URL: <http://wg21.link/P0581R1>

[P0829R0]

Ben Craig. [Freestanding Proposal](http://wg21.link/P0829R0). URL: <http://wg21.link/P0829R0>

[P1105R0]

Ben Craig; Ben Saks. [Leaving no room for a lower-level language: A C++ Subset](http://wg21.link/P1105R0). URL: <http://wg21.link/P1105R0>

[P1105R1]

Ben Craig; Ben Saks. [Leaving no room for a lower-level language: A C++ Subset](http://wg21.link/P1105R1). URL: <http://wg21.link/P1105R1>

[P1212R0]

Ben Craig. [Modules and Freestanding](https://wg21.link/p1212r0). 6 October 2018. URL: <https://wg21.link/p1212r0>